

A PROJECT REPORT ON

**ANOMALY DETECTION IN FILE SYSTEM
BASED ON USER ACCESS PATTERNS**

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Aditya More
Ashwin Taras
Riddhi Kulkarni

Exam No: B400050509
Exam No: B400050608
Exam No: B400050568



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled

Anomaly Detection in File System Based on User Access Patterns

Submitted by

Aditya More

Ashwin Taras

Riddhi Kulkarni

Exam No: B400050509

Exam No: B400050608

Exam No: B400050568

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Dr. K.C.Waghmare** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Dr. K. C. Waghmare
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
SCTR's Pune Institute of Computer Technology

Place: Pune

Date:

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Anomaly Detection in File System Based on User Access Patterns**’.*

*We would like to take this opportunity to thank **Dr. K. C. Waghmare** giving us all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

*We are also grateful to **Dr. G. V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support and suggestions.*

*In the end our special thanks to our industry mentor **Mr. Narendra Pal Singh**, Arctera.io (Formerly known as Veritas) and all faculty members for their whole hearted cooperation for completion of this report. We also thank our laboratory assistants for their valuable help in laboratory.*

Aditya More
Ashwin Taras
Riddhi Kulkarni

(B.E. Computer Engg.)

ABSTRACT

Enterprise networks and storage systems are becoming faster and more powerful. Recent advances in enterprise network infrastructure and data storage have greatly improved the scalability, performance, and accessibility of storing and retrieving data. However, these advancements also introduce new security risks in file access and in detecting unusual behavior in shared environments

This research employs the Network File System (NFS) as a proof of concept to demonstrate an anomaly detection method that combines unsupervised machine learning with a customized rule-based engine.

The system identifies anomalies such as unauthorized access, mass deletions, and irregular file modifications by analyzing system logs. The results are displayed through a custom user-friendly interface that allows real-time monitoring and notifications, allowing administrators to quickly respond to potential threats.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	3
1.3	Problem Definition and Objectives	4
1.3.1	Problem Statement	4
1.3.2	Problem Description	4
1.3.3	Objectives	5
1.4	Project Scope & Limitations	5
1.4.1	Project Scope	5
1.4.2	Limitations	6
1.5	Methodologies of Problem solving	6
2	Literature Survey	8
2.1	Literature Survey	9
3	Software Requirements Specification	12
3.1	Assumptions and Dependencies	13
3.1.1	Assumptions	13
3.1.2	Dependencies	13
3.2	Functional Requirements	13
3.2.1	Log Collection and Preprocessing	13
3.2.2	Role-Based Anomaly Detection	14
3.2.3	Anomaly Detection and Categorization	14
3.2.4	Alert Notification and Reporting	14
3.2.5	Dashboard and Visualization	15
3.2.6	Role and User Management	15
3.3	External Interface Requirements	15
3.3.1	User Interfaces	15
3.3.2	Hardware Interfaces	16
3.3.3	Software Interfaces	16
3.3.4	Communication Interfaces	17

3.4	Nonfunctional Requirements	17
3.4.1	Performance Requirements	17
3.4.2	Security Requirements	18
3.4.3	Software Quality Attributes	19
3.5	System Requirements	20
3.5.1	Database Requirements	20
3.5.2	Software Requirements (Platform Choice)	21
3.5.3	Hardware Requirements	21
3.6	Analysis Models: SDLC Model to be applied	22
3.6.1	Agile Framework Overview:	22
3.6.2	Project Phases and Timeline	22
4	System Design	24
4.1	System Architecture	25
4.2	Mathematical Model	26
4.3	Data Flow Diagram	28
4.4	Entity Relationship Diagram	30
4.5	UML Diagram	31
4.6	Sequence Diagram	32
4.7	Deployment Diagram	33
5	Project Plan	34
5.1	Project Estimate	35
5.1.1	Reconciled Estimates	35
5.1.2	Project Resources	36
5.2	Risk Management	37
5.2.1	Risk Identification	37
5.2.2	Risk Analysis	37
5.2.3	Overview of Risk Mitigation, Monitoring, Management	37
5.3	Project Schedule	41
5.3.1	Project Task Set	41
5.3.2	Task Network	43
5.4	Task Network	43
5.4.1	Timeline Chart	44
5.5	Team Organization	45
5.5.1	Team structure	45
5.5.2	Management reporting and communication	45
6	Project Implementation	47
6.1	Overview of Project Modules	48
6.2	Tools and Technologies Used	48

6.3	Algorithm Details	49
6.3.1	Isolation Forest	49
6.3.2	DBSCAN	50
6.3.3	K-Means Clustering	50
7	Software Testing	51
7.1	Type of Testing	52
7.2	Test Cases & Test Results	52
7.2.1	UI Testing (Frontend)	52
7.2.2	Server and API Testing	53
7.2.3	Machine Learning Engine Testing	54
7.2.4	Rule-Based Engine Testing	54
8	Results	56
8.1	Outcomes	57
8.1.1	Performance Metrics	57
8.1.2	Real-time Processing	57
8.1.3	System Improvement	58
8.1.4	User Feedback	58
8.2	Screen Shots	59
9	Conclusions	63
9.1	Conclusion	64
9.2	Future Work	64
9.3	Applications	65
10	Appendix	66
10.1	Appendix A: Configuration Details	67
	Appendix A: Configuration Details	67
10.1.1	System Configuration	67
10.1.2	Environment Variables	67
10.1.3	Sample Log Format	67
10.1.4	Explanation of Fields	68
10.1.5	Usage Notes	68
10.2	Appendix B: Publishing Details	69
10.2.1	Survey Paper	69
10.2.2	Implementation Paper	69
10.3	Appendix C: Plagiarism Report	70
11	References	71

List of Figures

4.1	System Architecture: Anomaly Detection System	25
4.2	System Architecture: Anomaly Detection System with Multiple Clients	26
4.3	State Transition Diagram for Mathematical Model	27
4.4	Data Flow Diagram (DFD-0)	28
4.5	Data Flow Diagram (DFD-1)	28
4.6	Data Flow Diagram (DFD-2)	29
4.7	Entity Relationship Diagram	30
4.8	Use Case Diagram	31
4.9	Sequence Diagram	32
4.10	Deployment Diagram	33
5.1	Phase 1: Research and Analysis	44
5.2	Phase 2: Implementation and Deployment	44
8.1	Landing Page	59
8.2	Sign-up Page	59
8.3	Dashboard: System Statistics	60
8.4	Alerts: Rule Based Engine	60
8.5	Alerts: ML Based Engine	61
8.6	Visualization Charts	61
8.7	Explainable AI (Summary of the Anomalies)	62
8.8	Example of Alert Email	62
10.1	Plagiarism Report	70

CHAPTER 1

INTRODUCTION

1.1 Overview

Businesses now mainly rely on Network Attached Storage systems to handle and store unstructured data due to the exponential growth of data in the current digital era. These systems are the foundation of data storage for big businesses, offering centralized, scalable, and manageable solutions for storing sensitive data, shared files, and business-critical information. However, as reliance on these systems increases, so does the challenge of properly monitoring and safeguarding them. Due to their inherent exposure to a variety of user activities and file operations, NAS systems are susceptible to security breaches, resource inefficiency, data corruption, and unauthorized access. Even though they are essential, traditional file and security monitoring tools are frequently insufficient to identify minute discrepancies in system activity or user behavior that might point to an anomaly.

Since numerous users access and edit files at once on NAS systems, it can be challenging to discern between safe user behavior and possible dangers. Furthermore, it is more difficult to spot unusual usage patterns in real time due to the massive scale of operations in large corporations. Conventional monitoring systems, for instance, miss opportunities to stop data breaches or system inefficiencies when they fail to detect abrupt increases in file changes, unexpected file deletions, or unusual access by a specific user. The high rate of false positives produced by traditional monitoring tools, which frequently inundate administrators with alerts about typical deviations in system usage, is a major problem in managing NAS file systems. This makes it challenging to identify actual anomalies. This increases the possibility of serious financial and reputational harm by fostering an environment where actual threats, like malicious activity or unauthorized access, can be disregarded.

There is an obvious need for a more intelligent, flexible system that can identify irregularities more accurately and consistently as businesses work to safeguard sensitive data and maintain system performance. By creating a sophisticated anomaly detection system designed especially for the NAS environment, this project seeks to address the aforementioned issues. The system will analyze a number of variables, including file change logs, audit logs, network usage, and system performance metrics like CPU and disk activity, using machine learning techniques like clustering and classification models.

By learning from common patterns of user behavior and system operations, the proposed solution will be able to recognize deviations that indicate anoma-

lies, including both internal and external threats. Installing such a system is crucial for large enterprises that depend on NAS for their data storage needs. By helping NAS administrators monitor, detect, and respond to suspicious activity more effectively, this solution is expected to enhance overall system security and performance.

1.2 Motivation

Advanced anomaly detection is becoming more and more necessary as managing Network Attached Storage (NAS) systems in large organizations becomes more complex. Large volumes of sensitive and important data are stored in these systems, and if not noticed in a timely manner, minor anomalies in user behavior, file access, or system behavior can have major repercussions. Administrators are frequently inundated with false alarms from traditional monitoring solutions, making it challenging to spot actual security threats or unusual activity in real time.

Several high-profile incidents underscore the importance of better anomaly detection mechanisms:

1. Sony Pictures Hack (2014): Unauthorized access to private data and unreleased films were made possible by hackers who breached Sony's file servers. The breach escalated because the unusual file access was not promptly detected.
2. Equifax Data Breach (2017): Attackers stole a great deal of private client data by taking advantage of file access flaws. By identifying unusual data access patterns, administrators might have been notified sooner and the harm might have been lessened.
3. Capital One Data Breach (2019): Unauthorized data access resulted from a misconfigured cloud storage system. The extent of the breach might have been reduced if it had been possible to spot odd file access and download activity.

These instances demonstrate how significant financial and reputational harm can result from anomalies that go unnoticed. In order to assist organizations in proactively identifying anomalous behavior, enhancing data security, and optimizing system performance, this project aims to develop an anomaly detection system that is based on machine learning and customized for the NAS environment. The solution uses sophisticated algorithms to reduce false

alarms and improve detection accuracy, making it a more dependable and effective enterprise NAS management tool.

1.3 Problem Definition and Objectives

1.3.1 Problem Statement

Create an NFS file server anomaly detection system that can recognize questionable user activity, detect anomalies automatically, find possible zero-day vulnerabilities in large-scale NFS systems, and offer an easy-to-use web interface.

1.3.2 Problem Description

Because of the complexity and size of operations in an enterprise setting, managing anomalies in NAS (Network Attached Storage) file systems is a special challenge. It gets harder to identify unusual user activity when several people are accessing and changing files at once. Static rules and threshold-based alerts, which are frequently used in traditional monitoring solutions, are insufficient to recognize subtle threats or anomalous behavior patterns. System administrators consequently often find it difficult to distinguish between benign changes in user behavior and actual security threats, which results in serious operational vulnerabilities.

The main issue is the absence of a practical, real-time solution that can offer insightful data about user behavior. Traditional techniques frequently produce a large number of false positives, flooding administrators with notifications that mask real abnormalities. For example, alerts that conceal potentially harmful activities, like insider threats or unauthorized data exfiltration, may be triggered by normal high-traffic file operations during peak business hours. This failure to recognize important warning indicators can have serious consequences, such as resource misuse, performance deterioration, and data breaches.

Furthermore, these conventional methods frequently fall short in adapting to changing user behavior, which makes them useless in dynamic settings where typical activity patterns may change over time. Because rule-based systems are static, they react slowly to emerging threats, exposing organizations to risks that might not meet predetermined detection criteria.

This project seeks to address these challenges by developing a hybrid machine learning and rule based engine anomaly detection system specifically designed for NFS. The system will leverage data from multiple sources, including:

- File change logs to track access trends and file operations.
- Audit logs to monitor security events and user behavior.
- Elasticsearch and Elastalert2 are used to create and examine anomalous logs in real time.
- A real-time notification system to notify the network administrator about the anomaly.
- System performance metrics such as CPU and disk usage, along with network packet statistics, to identify abnormal resource consumption.

1.3.3 Objectives

- To explore various insider-threats and anomalous activities related to NFS.
- To analyze existing systems and their limitations.
- To generate relevant dataset parallel to real-world scenario.
- To design Machine Learning algorithms and Rules for detection.
- To develop web-based interface with visualization dashboards.

1.4 Project Scope & Limitations

1.4.1 Project Scope

- Detect anomalies based on user access pattern.
- Provide real-time analysis of the file system activities.
- Log-based monitoring of multiple clients sharing NFS directories.
- Optimization using a blend of ML-algorithms and Rule-based engine.
- Provide insights about zero day vulnerabilities.

1.4.2 Limitations

- Application is limited to NFS (Network File System) as a Proof-of-Concept.
- Actual industry dataset is unavailable due to security policies, thus simulated dataset is used to measure performance.
- Log-based monitoring is operational on Client-side as tools for Server-side logging are insufficient for all the features.
- Operationality of the application will require more industry and company specific guidelines and use-cases to be implemented.

1.5 Methodologies of Problem solving

To effectively tackle the complex and evolving nature of threats in NAS environments, our system adopts a multi-faceted methodology. Each component contributes uniquely to enhance accuracy, interpretability, and adaptability in anomaly detection.

- **Explainable AI:**
Our AI bot analyzes previous system and user logs to generate concise summaries. This simplifies investigation for cybersecurity analysts by providing interpretable insights into potential threats and system behaviors.
- **Hybrid Detection Approach:**
We combine a rule-based engine with machine learning models to balance precision and adaptability.
 - The **Rule-based engine** captures known patterns and policy violations.
 - The **ML component** detects nuanced behavioral anomalies based on learned patterns.
- **Unsupervised Learning:**
By using unsupervised learning techniques, our system can detect:
 - Zero-day vulnerabilities, which are unknown and have no prior signatures.
 - Novel attack patterns that deviate from normal activity but may not match existing threat models.

This layered methodology ensures that our system is not only capable of detecting known threats but is also resilient and proactive in identifying emerging and previously unseen anomalies.

CHAPTER 2

LITERATURE SURVEY

2.1 Literature Survey

Sr. No.	Paper Name	Observation
1	Histogram-based Outlier Score (HBOS): A Fast Unsupervised Anomaly Detection Algorithm [1]	The Histogram-based Outlier Score (HBOS), a quick and effective unsupervised anomaly detection algorithm, is presented in this paper. HBOS models feature distributions using histograms, which enables it to identify anomalies without making assumptions about the data. It is a useful tool for real-time anomaly detection due to its computational efficiency, which makes it appropriate for large datasets.
2	Histogram-Based Anomaly Detection in Network Traffic [2]	The study suggests a histogram-based approach to network traffic anomaly detection. It provides a simple and effective method of anomaly detection by examining traffic patterns and utilizing histograms to identify deviations. This method is particularly helpful for real-time network monitoring.
3	Isolation Forest [3]	The authors introduce the Isolation Forest algorithm, a fast and effective unsupervised anomaly detection method. It isolates anomalies by recursively partitioning data, making it computationally efficient and scalable for large datasets, a key advancement in anomaly detection.
4	Unsupervised log message anomaly detection [4]	An unsupervised approach to anomaly detection in log messages is covered in this paper. By automatically identifying anomalous patterns in log data, the method improves system monitoring and security by detecting problems with system operations without the need for previously labeled data.

Sr. No.	Paper Name	Observation
5	Anomaly detection in temperature data using DBSCAN algorithm [5]	The DBSCAN algorithm is used in the paper to identify temperature data anomalies. DBSCAN is appropriate for detecting irregular patterns in time-series data since it efficiently detects abnormal temperature values by utilizing density-based clustering.
6	Flight operation anomaly detection based on one-class SVM [6]	A One-Class SVM is used in the study to detect anomalies in flight operations. By separating out anomalous flight behavior, this method offers a trustworthy model for spotting odd trends in aviation operations, which is essential for performance and safety monitoring.
7	LOF: Identifying density-based local outliers [7]	The authors present a density-based method for locating local outliers in datasets called the Local Outlier Factor (LOF). By comparing a data point's density to that of its neighbors, LOF determines how abnormal the data point is, which aids in identifying irregularities in intricate data distributions.
8	DeepLog: Anomaly detection and diagnosis from system logs through deep learning [8]	A deep learning-based method for identifying and analyzing irregularities in system logs is called DeepLog. An efficient tool for real-time anomaly detection in system logs, DeepLog detects departures from typical behavior by modeling system log patterns using LSTM networks.
9	LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs [9]	The study presents LogAnomaly, an unsupervised technique for identifying quantitative and sequential irregularities in unstructured logs. It offers a potent way to find problems in system log sequences without the need for labeled data by fusing pattern learning and anomaly detection.

Sr. No.	Paper Name	Observation
10	Self- attentive classification-based anomaly detection in unstructured logs [10]	The authors suggest a classification-based, self-attentive model for identifying irregularities in unstructured logs. The model improves anomaly detection accuracy for system monitoring by capturing intricate dependencies in logs through self-attention mechanisms.

Table 2.1: Literature Survey on Anomaly Detection Methods

CHAPTER 3

**SOFTWARE REQUIREMENTS
SPECIFICATION**

3.1 Assumptions and Dependencies

3.1.1 Assumptions

- The system will explicitly assign user roles and implement role-based access restrictions so that behavior can be compared based on these roles.
- The anomaly detection algorithm assumes that user role assignments and historical access data are reliable in order to establish a strong baseline for behavior comparison.
- In the environments where the system is installed, file server logs will already be accessible and enabled.

3.1.2 Dependencies

- Job definitions must be exact in order for the system to correctly categorize user behaviors and detect relevant abnormalities.
- Hardware infrastructure, cloud services (if used), and software environments supporting the detection models, analysis, and role management are examples of dependencies.
- The effectiveness of the system depends on its proper configuration, which includes role-based rules and thresholds for categorizing anomalies and severity levels.

3.2 Functional Requirements

3.2.1 Log Collection and Preprocessing

- Gather logs from Linux systems continuously.
- Gather crucial log data, such as the IP address, timestamp, file path, user name, file accessed, and operation (read, modify, or remove).
- It is necessary to parse logs and standardize them into a processing format.
- Ensure that logs are safely transferred and kept for analysis.
- Handle missing or corrupted logs and generate the appropriate alerts.

3.2.2 Role-Based Anomaly Detection

- Give the designated user roles (Admin, Regular User, etc.) the proper access permissions.
- Analyze access patterns in light of the user's role, keeping an eye out for any anomalies that might raise concerns.
- To progressively increase detection accuracy, create role-based user access profiles.

3.2.3 Anomaly Detection and Categorization

- You can spot irregularities by comparing recent user behavior to historical patterns.
- Identify unusual access patterns, such as unauthorized file access, unusual file locations, or unusual times.
- Assign a severity level (Low, Medium, or High) to each anomaly discovered in accordance with established guidelines.
- Permit administrators to modify severity thresholds in accordance with the organization's needs.
- Learn continuously from new data to improve the performance of detection algorithms.

3.2.4 Alert Notification and Reporting

- When anomalies are found, create recurring alerts that are categorized by users and severity.
- Notify assigned security staff members through third-party tools, the system dashboard, or email.
- Create thorough reports of anomalies that include timestamps, user information, actions, and severity levels.
- Also include a summary of entire report for easy auditing and reporting with the help of explainable AI
- Maintain a record of previous audit logs for upcoming studies and system performance assessments.

3.2.5 Dashboard and Visualization

- Provide an interactive dashboard to monitor anomaly detection, log collection, and system health.
- Offer filters to view anomalies according to user role, severity, and time range.
- Provide visual representations of the discovered anomalies, including timelines, trends, and access patterns.
- Give administrators the ability to investigate anomalies in greater detail.

3.2.6 Role and User Management

- Assign users the appropriate access rights so they can add, modify, and remove user roles.
- Verify that role changes are reflected in access control and anomaly detection.
- Give security administrators permission to manage user accounts (adding, deleting, and altering them) and implement role-based access controls.

3.3 External Interface Requirements

3.3.1 User Interfaces

- **Web-based Admin Console:** The system will feature a web-based interface accessible via browsers. This interface will allow administrators to monitor real-time anomaly alerts, visualize usage patterns, and manage system configurations.
- **Authentication Interface:** Users will be required to log in through a secure authentication mechanism. Only authorized personnel will be able to view or configure sensitive data thanks to the implementation of role-based access control, or RBAC.
- **Log Insights Dashboard:** A visual dashboard will present summarized log data, anomaly trends, and system health metrics, using interactive graphs and tables to facilitate decision-making.

- **Alert and Notification Panel:** Users will receive alerts through the UI for high-risk anomalies or system issues. This panel will also allow them to acknowledge, ignore, or tag alerts for future reference.

3.3.2 Hardware Interfaces

- **Linux File System:** The system must interface with the Linux file system in order to collect activity logs. To find any irregularities, the system will analyze the necessary access logs from the servers.
- **Storage Devices:** The system should be able to communicate with network-attached storage (NAS) devices running Linux as well as local devices. It will monitor unstructured data access on these storage systems and spot anomalies in file operations.
- **Network Infrastructure:** The system will connect to the business's network infrastructure in order to collect logs and securely transmit data for centralized processing. If necessary, this may involve monitoring network traffic, switches, and routers to spot anomalies in the methods by which data is accessed across networked storage.

3.3.3 Software Interfaces

- **Linux Operating System:** The system will interact with the Linux file system in order to gather access logs. It can be used with system logs (/var/log/syslog), audit logs (/var/log/audit/audit.log), or custom Linux security logs related to file access and user activity.
- **Database Systems:** The system will interface with a database that contains the collected logs, processed anomaly detection results, and historical data. The database will support the role-based anomaly detection mechanism and store user behavior profiles for later analysis.
- **Visualization Tools:** The system dashboard, which visualizes anomalies and system health, will interface with data visualization libraries to present insights graphically. In order to render graphs, charts, and other visualizations, this may require integrating with applications such as Chart.js, Grafana, Plotly, or D3.js.

3.3.4 Communication Interfaces

3.3.4.1 Log Collection APIs

The system will interact with Linux logging APIs and services, including:

- iNotify methods for gathering Linux server logs in real time.
- Access security-related logs, including file access, permission changes, and user activity, by auditing utilities.

3.3.4.2 Alerting and Notification APIs

The system will send real-time alerts and notifications using APIs, such as:

- Email utilities for sending alert notifications by email (e.g., Google SMTP).
- Using Elastalert2 to send alerts and messages in real time

3.3.4.3 Security and Authentication APIs

To manage authentication and role-based access controls, the system will communicate with security APIs like OAuth or LDAP. For both users and system administrators, this guarantees safe access and appropriate authorization.

3.3.4.4 RESTful APIs for Anomaly Data

The system will provide RESTful APIs so that external security systems can access anomaly detection results. These APIs will enable integration with external tools that need to retrieve anomalous data or perform further analysis.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

- **Real-Time Log Processing:** Logs from the Linux file servers must be processed almost instantly in order for the system to detect anomalies promptly. Large volumes of logs should be processed efficiently by the system, ensuring that anomalies are found seconds after they occur.

- **Scalability:** The system needs to be scalable in order to handle big environments with lots of Linux servers and high log volumes. To sustain performance as the business grows, the system should scale either horizontally (by adding more processing units) or vertically (by increasing processing capacity).
- **Throughput and Latency:** With a high throughput, the system should be able to process thousands of log entries per second. A few seconds should be the maximum amount of time that passes between collecting log entries and detecting anomalies in order to guarantee prompt alerts for security incidents.
- **Resource Utilization:** The system needs to make the best use of its CPU, memory, and storage. The system should be able to adapt to different workloads and ensure that it doesn't overload the servers where it is installed, particularly in environments with limited computing resources.
- **Fault Tolerance and Recovery:** The system's design ought to include fault tolerance. In the event of failures (like system crashes or network issues), the system must be able to recover without losing data, either by itself or with minimal help. Installing redundant systems and regularly backing up log data are two ways to ensure uptime.
- **Response Time for Alerts:** System-generated alerts must be sent to security personnel within five seconds of the anomaly being discovered. Response time is crucial for high-severity incidents in order to facilitate timely risk mitigation actions.
- **System Load Handling:** The system must perform optimally in a variety of load scenarios, including periods when the file server may be busy. Performance testing should be done to ensure that the system can tolerate stressful conditions without suffering a drop in detection speed or accuracy.

3.4.2 Security Requirements

- **Data Security:** All log data collected from the Linux file servers must be transmitted and stored safely by the system. To prevent manipulation or unauthorized access, data must be encrypted both in transit and at rest using methods like AES-256.

- **Role-Based Access Control (RBAC):** To ensure that only authorized users, like security administrators and analysts, can view anomalies, configure detection parameters, and access specific features, the system must implement strict role-based access control. Unauthorized users shouldn't have access to sensitive logs or system data.
- **Authentication and Authorization:** The system should interface with existing security frameworks (like LDAP and OAuth) to authenticate users and grant access based on predefined roles. Strong password policies, session timeout mechanisms, and multi-factor authentication (MFA) should all be implemented to safeguard user accounts.
- **Audit Logging:** Every action performed within the system, including configuration changes, login attempts, and actions taken to resolve anomalies, should be meticulously documented. These logs should be protected from manipulation and available for security reviews.
- **Anomaly Data Protection:** The detected anomalies and their accompanying metadata (such as user actions, file names, and timestamps) must be protected from unauthorized access. The system should include access controls to ensure that only authorized personnel can view, edit, or archive anomaly-related data.
- **Intrusion Detection and Prevention:** It is necessary for the system to be immune to intrusions. It should include mechanisms for detecting and responding to intrusions, such as unauthorized attempts to access the system's logs or modify its detection algorithms. Regular security audits should be conducted to ensure the system's integrity.

3.4.3 Software Quality Attributes

- **Reliability:** The system must consistently detect anomalies with minimal false positives or negatives. It should remain operational under expected workloads and gracefully handle unexpected inputs or system errors.
- **Scalability:** Without sacrificing performance, the architecture should be scalable to accommodate growing amounts of log data from numerous NAS servers and user endpoints.
- **Maintainability:** The system should be modular and well-documented to facilitate future enhancements, debugging, and integration of new detection models or rule sets.

- **Performance:** Real-time or near-real-time processing of logs and generation of alerts is critical. The system should optimize resource usage for efficient analysis of large datasets.
- **Security:** The platform must ensure secure data handling, with encrypted transmission of logs and access controlled by strong authentication and authorization mechanisms.
- **Usability:** The admin interface should be intuitive, allowing cybersecurity analysts to easily navigate dashboards, investigate anomalies, and configure system parameters without deep technical knowledge.

3.5 System Requirements

3.5.1 Database Requirements

- **Type of Database:** The system should use a relational database such as PostgreSQL or MySQL to store collected logs, anomaly detection results, and historical access patterns. The choice of a relational database allows for structured queries and scalable storage solutions.
- **Data Storage Capacity:** The database must have the capacity to hold large volumes of log data, role-based access patterns, and anomaly metadata for extended periods of time. Although a 1 TB initial capacity is recommended, with room to grow as log data increases, the minimum storage requirement should be determined using the expected log volume.
- **Data Retention Policy:** A retention policy that archives or purges older logs (such as those that are more than six months old) should be implemented in compliance with organizational policies. Anomalies and their fixes should be retained for a longer time for auditing and security analysis.
- **Backup and Recovery:** In the event of corruption or system failure, the database must be able to quickly recover and support frequent backups. Automated daily backups with off-site storage options should be configured to ensure data availability.

3.5.2 Software Requirements (Platform Choice)

- **Operating System:** The system must be powered by Linux, especially those that are reliable, secure, and commonly used in business settings. Tools such as Syslog are useful for collecting and analyzing logs from Linux systems in real time.
- **Log Collection:** For collecting security and file access logs from Linux client systems, we use Python iNotify tools and Elasticsearch APIs.
- **Backend:** The Python backend was constructed using Django, Flask, and FastAPI, and it was integrated with a relational database. It also featured event-based APIs for alerts and notifications.
- **Frontend:** React.js for building the user interface with styling in Tailwind.
- **Database:** As previously stated, relational databases like MySQL, PostgreSQL, SQLite, etc. will be the database of choice.
- **Programming Languages:** SQL, Typescript/Javascript [for UI], Python.
- **Anomaly Detection Libraries:**
 - Scikit-learn: For putting into practice anomaly detection models based on machine learning.
 - Pandas and NumPy: For managing big datasets and preprocessing data.
- **Visualization Libraries:**
 - Plotly or Matplotlib: For generating visual insights into detected anomalies.
 - Grafana or Kibana: For real-time dashboards (optional but highly recommended).

3.5.3 Hardware Requirements

- **Additional Storage:** For archiving older logs and backups in environments with high data volumes or long retention periods, Network Attached Storage (NAS) or cloud-based storage options (such as Amazon S3 or Google Cloud Storage) should be available.

- **Backup and Redundancy:** In order to guarantee high availability in the event of server failure, backup servers ought to be configured. To avoid downtime, redundant power supplies, cooling, and network connectivity are advised.

3.6 Analysis Models: SDLC Model to be applied

The Agile methodology will be used in the development of the anomaly detection system, which will be divided into two primary stages: implementation and deployment (October–December 2024) and research and analysis (August–September 2024). Agile’s iterative methodology, which incorporates frequent feedback and ongoing enhancement, is perfect for managing the dynamic nature of the system’s needs.

3.6.1 Agile Framework Overview:

Through 2-week sprints, the project will advance and produce small, noticeable improvements. In this Agile setup, roles include:

- **Product Owner:** Assigns tasks a priority based on stakeholder demands.
- **Scrum Master:** Ensures that the team adheres to Agile practices.
- **Development Team:** In charge of testing and feature implementation.

3.6.2 Project Phases and Timeline

The goal of each of the two project phases is to progressively build, test, and enhance the anomaly detection system.

3.6.2.1 Phase 1: Research and Analysis (Aug-Sep 2024)

- **Research Activities:**
 - Methods Research and Literature Review to identify the best tools and techniques for anomaly detection
- The goal of each of the two project phases is to progressively build, test, and enhance the anomaly detection system.

- putting together conditions that outline the system's requirements, such as role-based access controls and classification of anomaly severity.
- Data sources and infrastructure configuration to create the setting for log gathering and analysis.

- **Analysis and Development:**

- Data is collected from Linux file systems in order to establish baselines.
- To begin anomaly detection, logs are preprocessed and an ML algorithm is configured.
- The creation and training of the machine learning model.
- To ensure the system works correctly with the initial data, testing and integration are required.
- User input from stakeholders was obtained to enhance the detection model and user interface.

3.6.2.2 Phase 2: Implementation and Deployment (Oct-Dec 2024)

- **Implementation:**

- Model refinement and monitoring setup based on Phase 1 insights.
- Developing and testing an alert system to provide real-time anomaly notifications.
- Dashboard creation for anomaly classification and system health visibility.

- **Deployment:**

- To improve the system, use activity tracking, system validation, and user interface refinement.
- Technical details and user instructions are documented.
- The live system's final review, deployment, and maintenance.
- Post-deployment system monitoring to guarantee ongoing functionality and future enhancement planning.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

This is a high-level system overview. The data flow and the implementation of anomaly detection using NFS in a single-user scenario are depicted in the diagram. It offers a broad comprehension of the architecture and its essential elements.

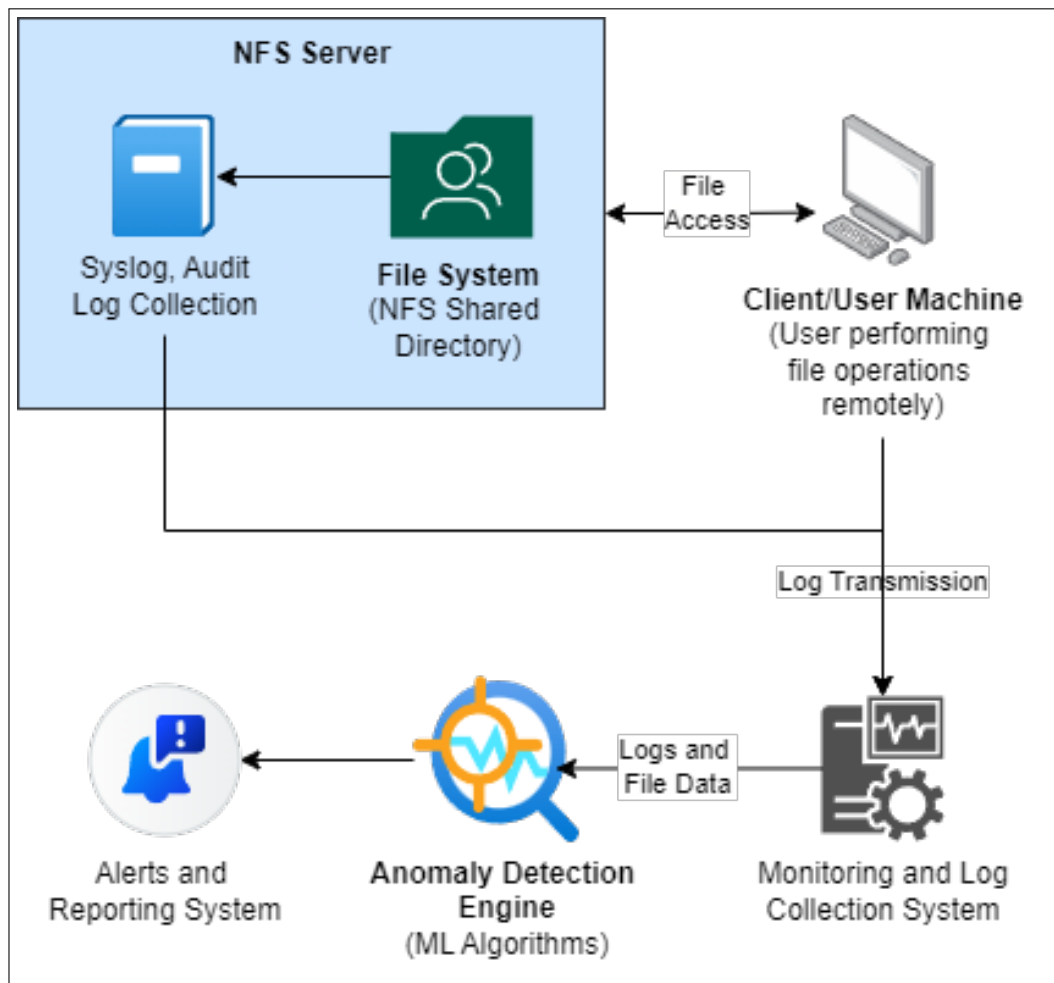


Figure 4.1: System Architecture: Anomaly Detection System

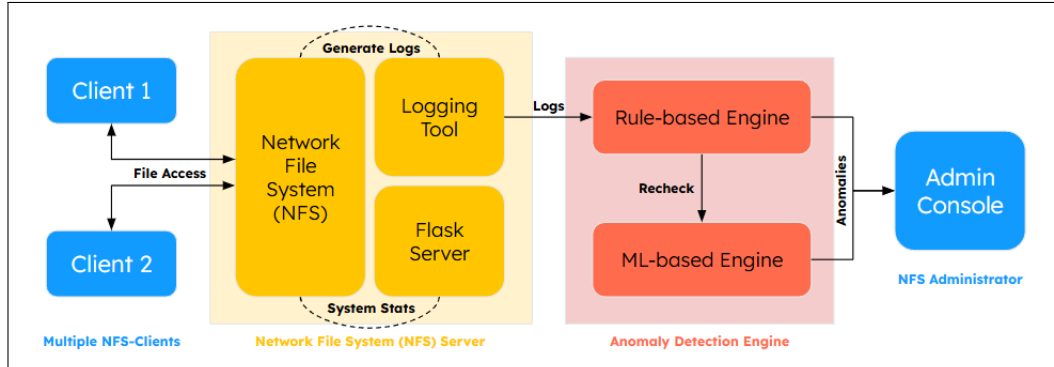


Figure 4.2: System Architecture: Anomaly Detection System with Multiple Clients

4.2 Mathematical Model

The Anomaly Detection System can be formally described using a mathematical model as follows:

$$\mathbf{S} = \{\mathbf{I}, \mathbf{P}, \mathbf{O}, \mathbf{F}\}$$

Where,

- **I** = Set of Inputs
 - Access logs from NAS file servers
 - Network traffic logs
 - User commands and configurations
- **P** = Set of Preprocessing Functions
 - Parsing log data
 - Normalization and feature extraction
- **O** = Set of Outputs
 - Alerts for detected anomalies
 - Reports and visualizations for administrator
- **F** = Set of Core Functionalities
 - **F₁**: Rule-based anomaly detection

- **F₂**: ML-based anomaly detection (unsupervised learning)
- **F₃**: Alert generation and reporting

Thus, the system can be formally represented as:

$$S : (I) \xrightarrow{P,F} O$$

meaning that the system accepts input data (I), applies preprocessing (P) and core functionalities (F), to generate the desired output (O).

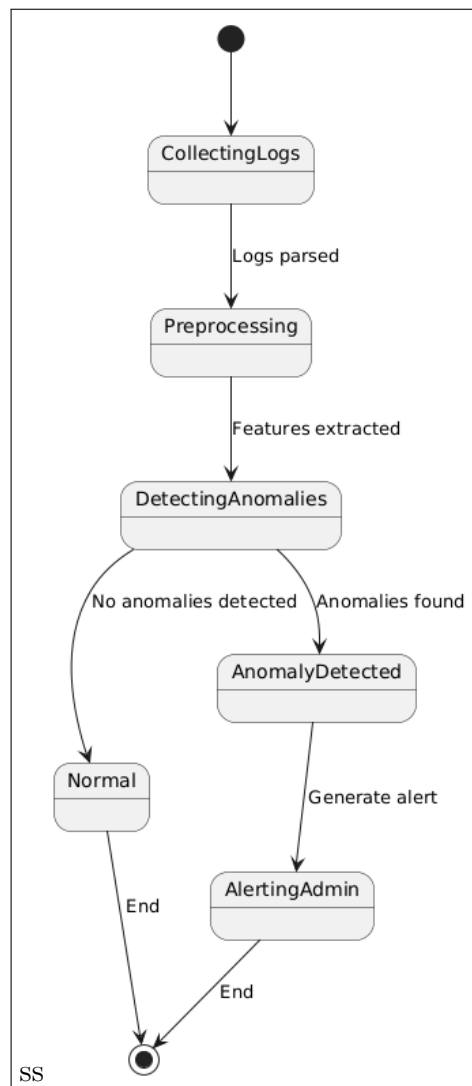


Figure 4.3: State Transition Diagram for Mathematical Model

4.3 Data Flow Diagram

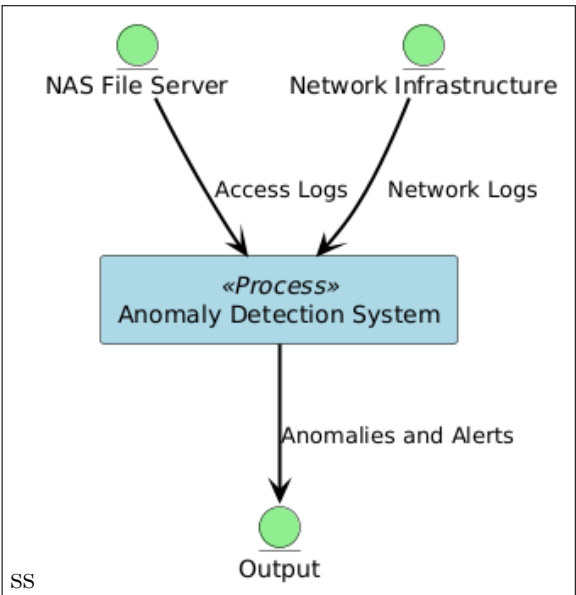


Figure 4.4: Data Flow Diagram (DFD-0)

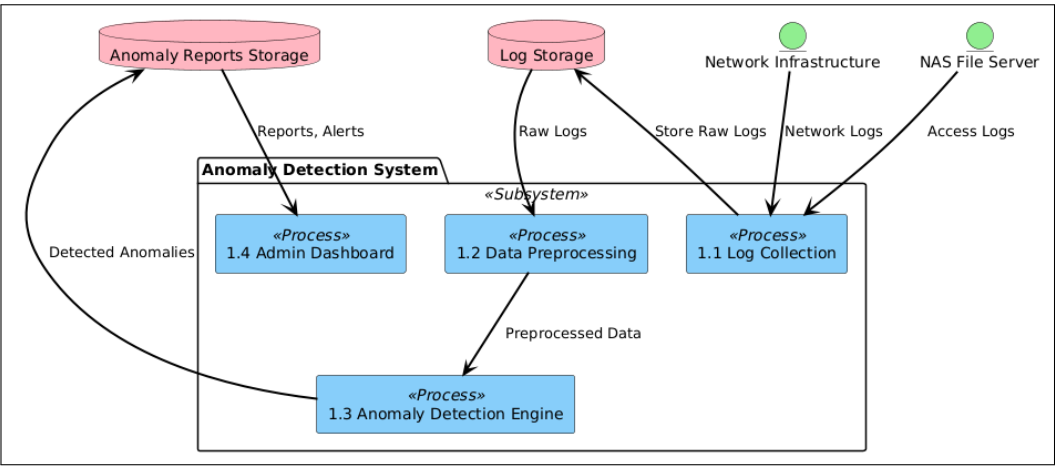


Figure 4.5: Data Flow Diagram (DFD-1)

- **DFD-0:** Entire system as one block
- **DFD-1:** Break into major modules: Log Collection, Preprocessing, Detection, Dashboard
- **DFD-2:** Zoom into how anomaly detection itself works internally

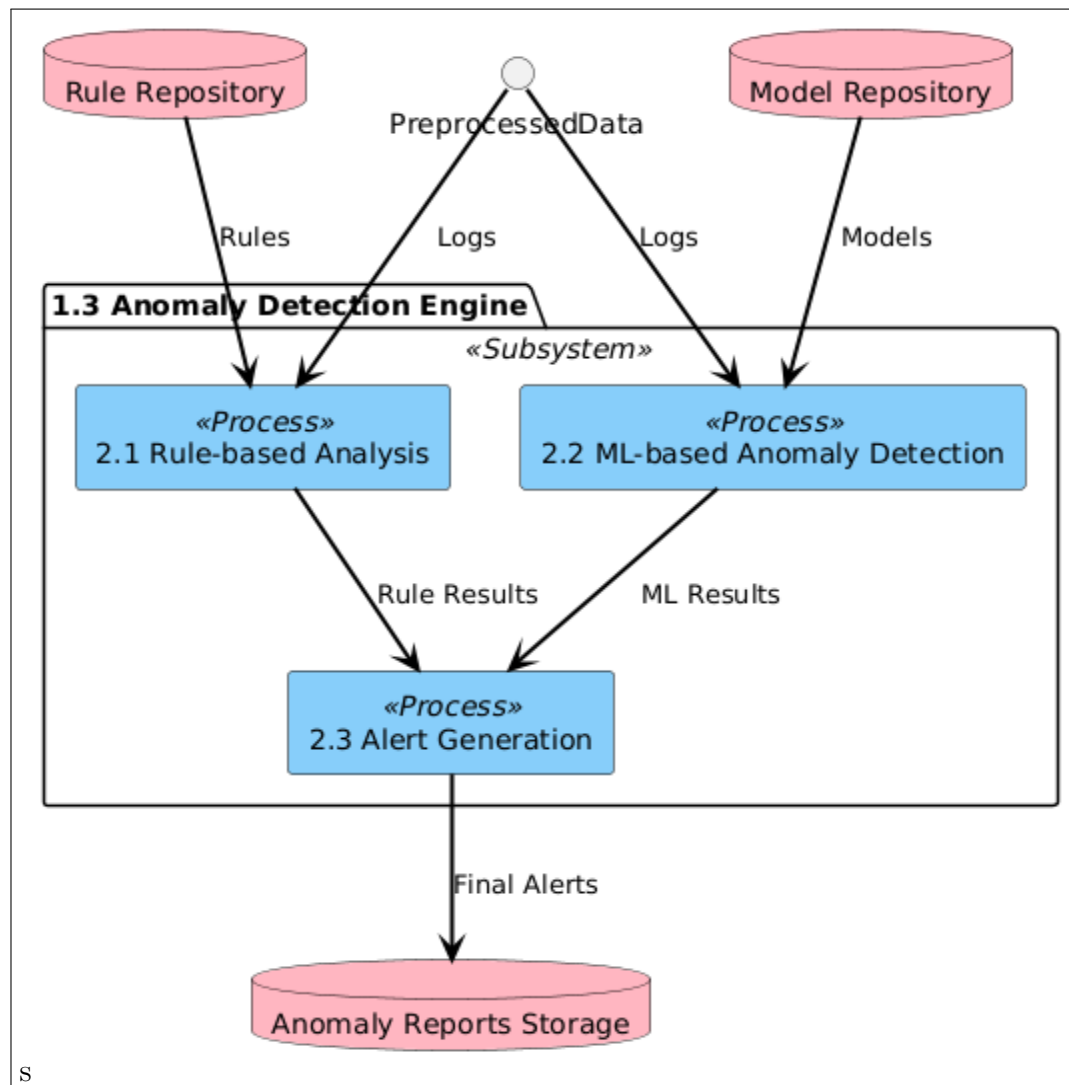


Figure 4.6: Data Flow Diagram (DFD-2)

4.4 Entity Relationship Diagram

Here, we have shown how various entities will interact using the system architecture.

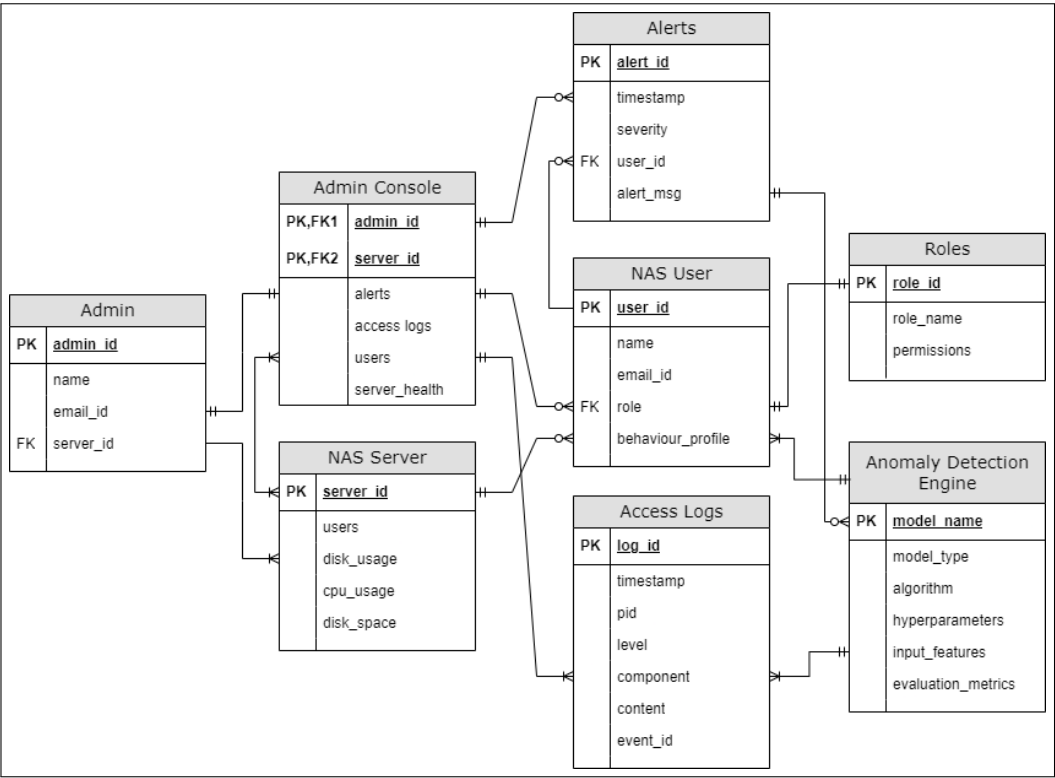


Figure 4.7: Entity Relationship Diagram

The relationship structure and connections between the important entities in the system, including the Admin Console, NAS user, NAS server, and access logs, are shown by the Entity-Relationship Diagram (ERD). The relationship between these data entities is depicted in the diagram as an interconnected network.

4.5 UML Diagram

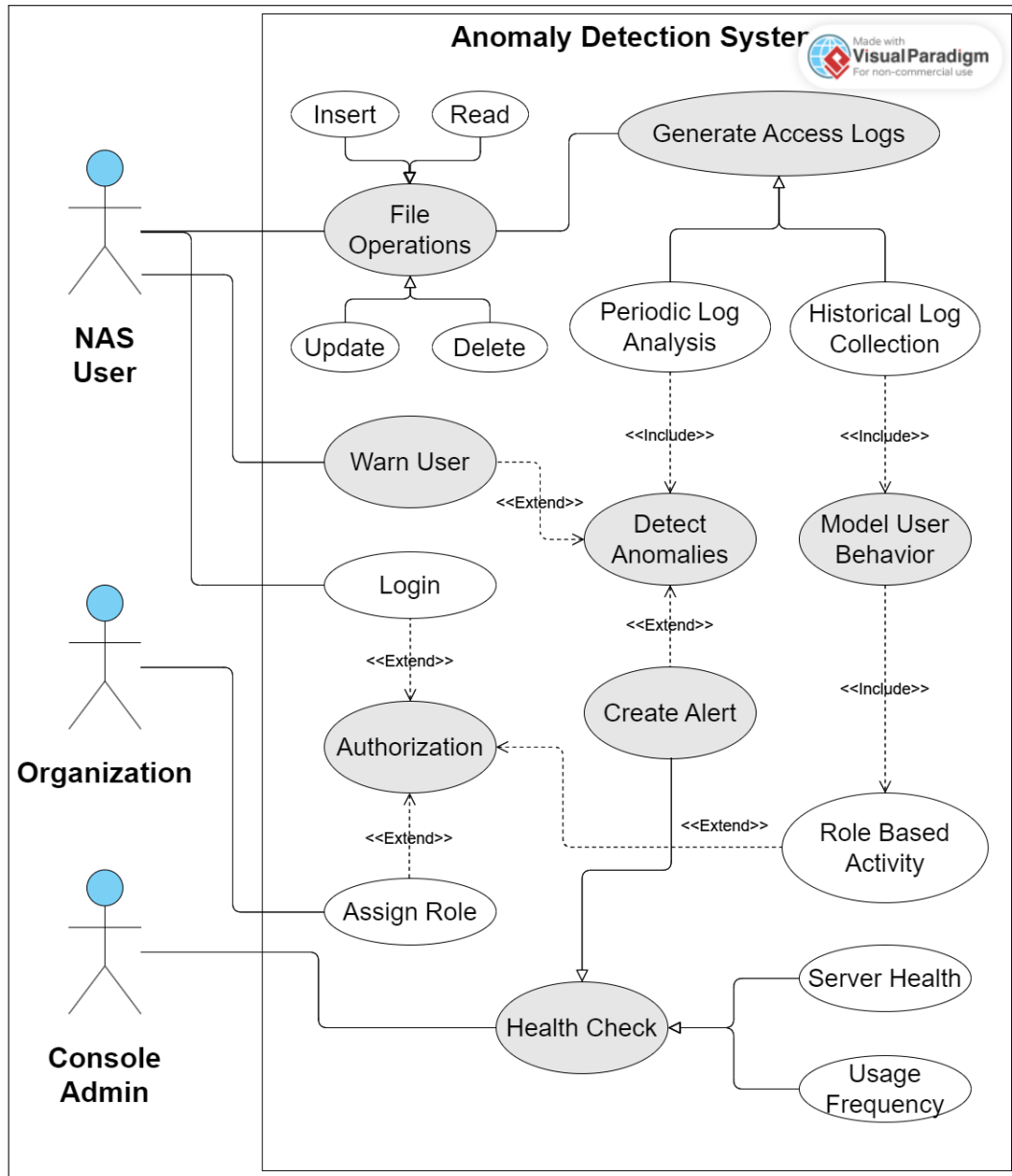


Figure 4.8: Use Case Diagram

4.6 Sequence Diagram

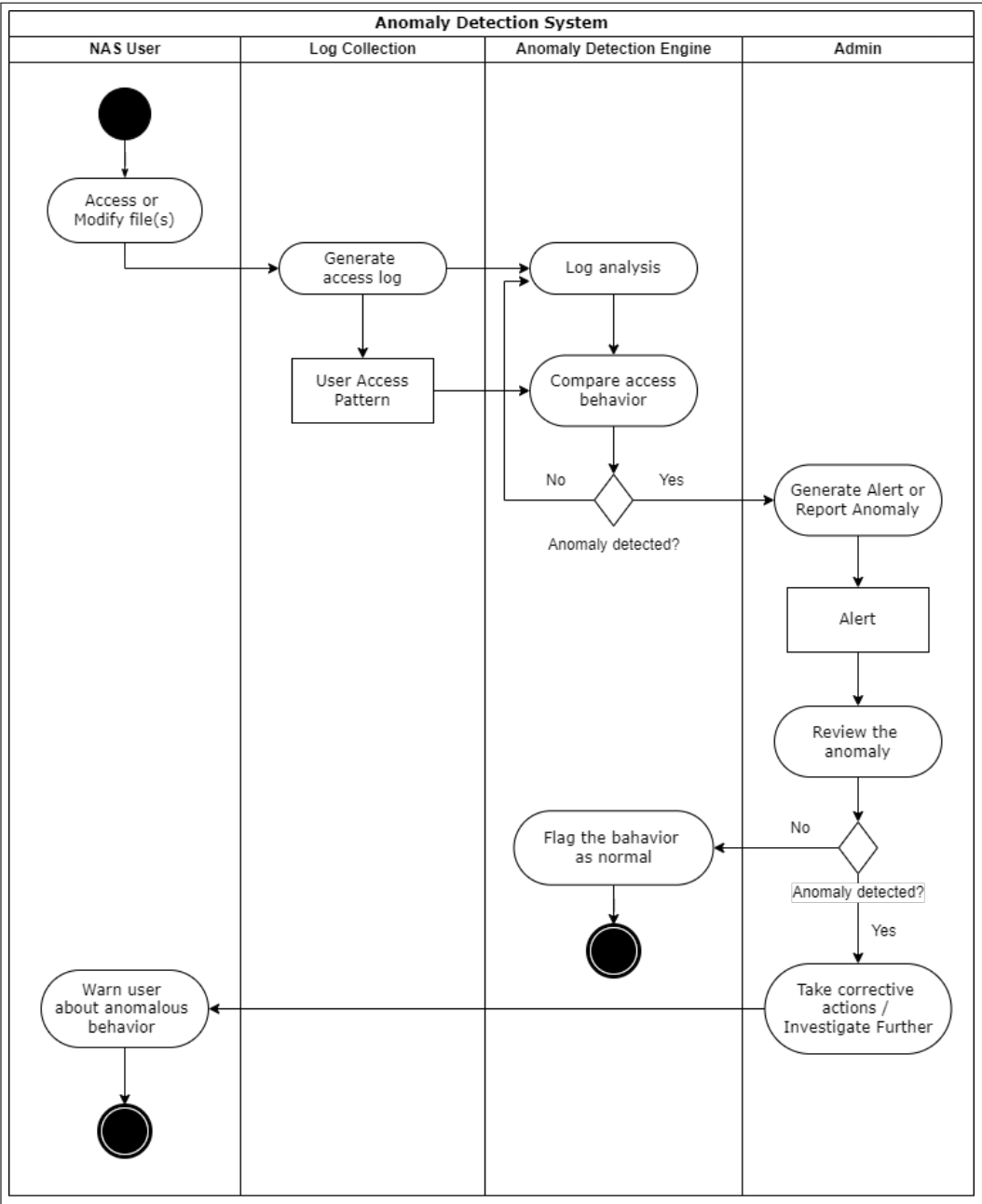


Figure 4.9: Sequence Diagram

4.7 Deployment Diagram

Low latency and secure access to NAS resources are guaranteed by the deployment of the anomaly detection system on a Flask-based server situated within the company's network. Both the RESTful/WebSocket APIs that communicate with client apps and the machine learning anomaly detection model are hosted on the Flask server. The Elasticsearch service connected to the Flask server houses the processed data and anomalies found.

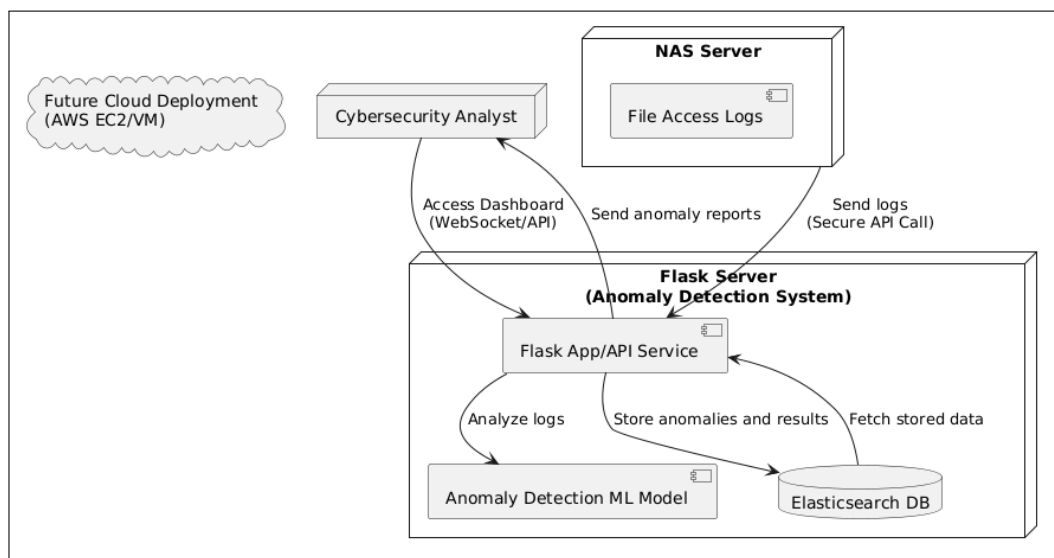


Figure 4.10: Deployment Diagram

A Web Dashboard, which retrieves real-time anomaly updates, user activity statistics, and system health information through secure API calls and WebSocket connections, is how cybersecurity analysts (such as NAS administrators) communicate with the system.

Because of the architecture's scalability, future migrations to cloud infrastructure, such as AWS EC2 instances, can be made with minimal modifications.

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- **Design Phase: 4-6 weeks**
 - Focus on understanding the system architecture, designing the components (Flask server, machine learning model, API integration, and Elasticsearch setup).
- **Development Phase: 8-12 weeks**
 - Backend Development (Flask API and server setup): 4 weeks
 - Machine Learning Model Development (training and integrating the anomaly detection model): 6 weeks
 - Elasticsearch Integration (setting up and testing Elasticsearch for data storage): 2 weeks
- **Testing Phase: 2-3 weeks**
 - Unit Testing and Integration Testing: 1 week
 - Anomaly Detection Validation (ensuring the model detects known and unknown threats accurately): 1 week
 - Performance Testing (ensuring the system scales with increasing data volumes): 1 week
- **Deployment Phase: 2-3 weeks**
 - Local Deployment: 1 week
 - Cloud Migration Setup (future): 1 week
 - Final System Optimization: 1 week
- **Risk Mitigation Adjustments: 1-2 weeks**
 - Extra buffer for potential delays due to integration or model issues.

5.1.2 Project Resources

- **Human Resources:**

1. Software Developers: Responsible for frontend development using Next.js and backend development using Flask and integration with the machine learning model.
2. Machine Learning Specialist: Responsible for the anomaly detection model, training, and validation.
3. Cybersecurity Analyst: Ensures that the system meets security standards and helps in validating anomaly detection.
4. Project Manager: Oversees the project, ensuring timelines, resources, and tasks are on track.

- **Technical Resources:**

- **Servers:**

- * Local NFS Servers: For data collection and testing during development.
- * Cloud Infrastructure (AWS EC2 or VM): For future deployment and scalability.

- **Software:**

- * Flask: For backend server and API development.
- * Machine Learning Libraries: for building the anomaly detection models.
- * Elasticsearch: For storing and querying log data.
- * Docker: For containerizing the system to ensure easy deployment and scalability.
- * Git: For version control and collaboration.

- **Tools for Monitoring and Performance:**

- * Web Console: For monitoring system performance and anomaly alerts.
- * Postman: For testing API calls and their performance.
- * Elastalert: For email alerts regarding anomalous behavior.

5.2 Risk Management

Identification, analysis, and planning of responses to possible threats that could compromise the anomaly detection system's performance or success are all part of risk management. Throughout the course of the project, it guarantees the system's resilience, dependability, and maintainability.

5.2.1 Risk Identification

We have identified potential risks in the project and they are as follows:

1. Failures in real-time log collection brought on by service or network outages.
2. Unknown anomalies are not detected by the machine learning model.
3. Excessive rule-based detection false positives.
4. The web dashboard is not correctly updating or rendering logs.

5.2.2 Risk Analysis

Every risk that has been identified has been examined according to its category, impact severity, and likelihood of occurrence:

- **High Impact + Medium Probability:** False positives and unsuccessful log collection.
- **Critical Impact + Low Probability:** Anomalies are not detected by the ML model.
- **Moderate Impact + Low Probability:** Failure of the dashboard visualization.

Appropriate analysis aids in setting priorities for proactive monitoring and mitigation techniques.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- The following tables provide detailed risk mitigation strategies:

Table 5.1: Risk ID 1: Log Collection Failure

Risk	Failure in Real-Time Log Collection
Category	Integration/Deployment
Probability	Medium
Impact	High
Response	Mitigate
Strategy	1. Implement retry logic and persistent logging in Flask server. 2. Use buffering/message queues like Redis to prevent data loss. 3. Monitor system performance and auto-restart logging agent on failure.
Status	Identified

Table 5.2: Risk ID 2: ML Model Inaccuracy

Risk	ML Model Fails to Detect Anomalies
Category	Machine Learning
Probability	Low
Impact	Critical
Response	Mitigate
Strategy	1. Use ensemble model approach (Isolation Forest, DBSCAN, K-Means). 2. Regularly retrain model with new data. 3. Validate with unsupervised evaluation metrics.
Status	Identified

Table 5.3: Risk ID 3: False Positives from Rules

Risk	Rule-based Engine Producing False Alarms
Category	Rule Based Engine
Probability	Medium
Impact	Moderate
Response	Mitigate
Strategy	1. Fine-tune thresholds and logical conditions. 2. Correlate alerts with ML output to improve confidence. 3. Apply rule validation based on historical logs.
Status	Identified

Table 5.4: Risk ID 4: Dashboard Render Failure

Risk	Web Dashboard Not Rendering Logs Properly
Category	Frontend/Web Dashboard
Probability	Low
Impact	Moderate
Response	Mitigate
Strategy	1. Add fallback UI for missing or delayed data. 2. Enable paginated/lazy loading in React frontend. 3. Implement API and frontend error logging.
Status	Identified

5.3 Project Schedule

5.3.1 Project Task Set

To guarantee the anomaly detection system was developed, integrated, and validated step-by-step, the project was broken up into a structured set of tasks. The following are the main tasks completed:

1. Literature Survey and Requirement Analysis

- Examined log monitoring systems and anomaly detection methods.
- Discovered flaws in the current NFS insider threat detection tools.

2. System Design

- NFS, logging agents, an ML engine, rule-based detection, an alerting system, and a dashboard comprised the designed modular architecture.
- Outlined interface requirements, data flow, and component interactions.

3. Prototype Setup: NFS Server and Client Configuration

- Configured an NFS server on a Linux Virtual Machine on a Windows host.
- Set up multiple clients using Docker to access shared directories via NFS, connecting multiple computers to the same network.
- Verified file access, permissions, and sharing behavior between nodes.

4. Dataset Preparation

- Python and iNotify tools with the Elasticsearch API were used to log user actions and file system activity.
- IP address, file path, operation type, and timestamps are among the extracted fields.
- Pre-processed and encoded features suitable for unsupervised machine learning algorithms.

5. Model Development and Evaluation

- Implemented Isolation Forest, DBSCAN, and K-Means clustering algorithms on the given datasets.
- Developed ensemble learning logic to merge multiple anomaly detection outputs to give efficient and accurate results.
- Silhouette score and Davies-Bouldin score were used as performance measures for the ML engine.

6. Rule-Based Detection System

- Created static rules for actions like unauthorized access, suspicious extensions, and mass deletions.
- Put in place a rule engine to handle logs instantly. Also added real time alerts system via elasticsearch

7. Backend and API Development

- Built a backend using Flask to manage anomaly analysis and log ingestion as well as for real time logging via WebSocket Technology.
- ML and rule engines were connected via APIs.

8. Frontend Dashboard Development

- Created a front-end using React.js to show system statistics and anomalies.
- Real-time alerts, filters, and interactive charts were implemented.

9. System Integration

- Integrated dashboard, detection engines, log collection, and NFS setup.
- Made sure that all data flowed smoothly from user file activity to alerting and detection.

10. Testing and Validation

- All modules underwent integration and unit testing.
- Using both simulated and actual file access patterns, the detection accuracy was verified..

11. Deployment and Documentation

- Docker was used to containerize the system for reproducibility.
- Created the final project report, architecture documentation, and user manual for setup and integration.

5.3.2 Task Network

5.4 Task Network

The logical ordering and dependencies between the different tasks in the project are visible thanks to the task network. The task relationships are shown in the following table.

Table 5.5: Task Dependency Table

Task	Depends On
Literature Survey and Requirement Analysis	– (Initial Task)
System Design	Literature Survey
NFS Server and Client Setup	System Design
Dataset Preparation	NFS Setup
Model Development and Evaluation	Dataset Preparation
Rule-Based Detection System	Dataset Preparation
Backend and API Development	Model + Rule-Based System
Frontend Dashboard Development	System Design
System Integration	Backend, Frontend, Detection Systems
Testing and Validation	System Integration
Deployment and Documentation	Testing and Validation

5.4.1 Timeline Chart

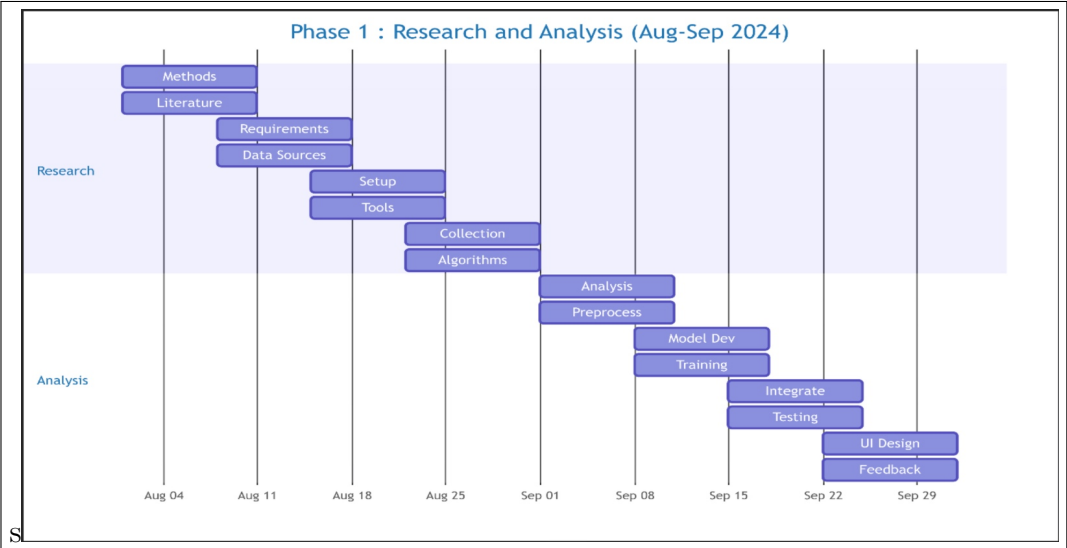


Figure 5.1: Phase 1: Research and Analysis

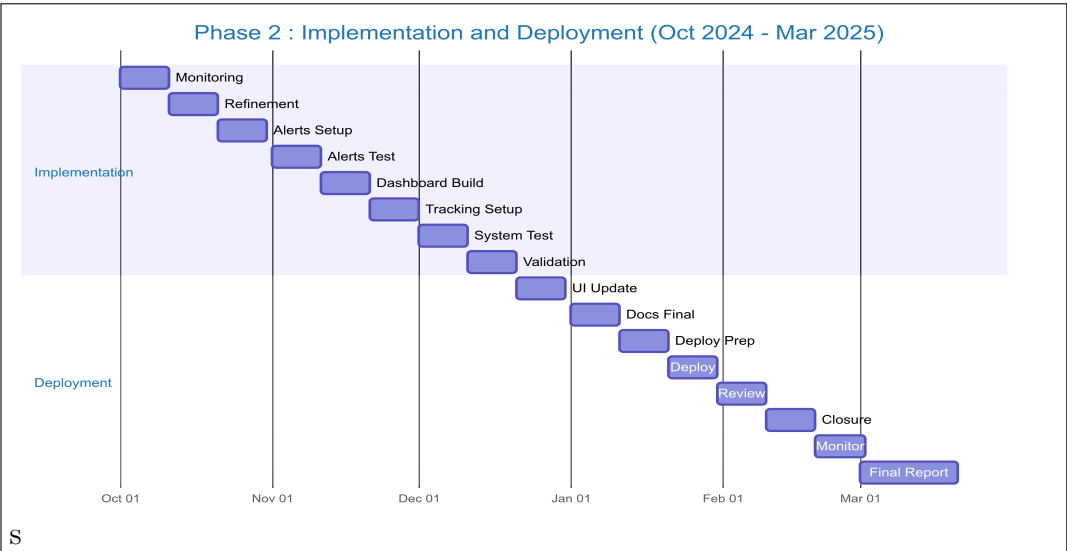


Figure 5.2: Phase 2: Implementation and Deployment

5.5 Team Organization

5.5.1 Team structure

- **Aditya More:** Backend, Python and DevOps.
- **Ashwin Taras:** Frontend, System Design and Authentication
- **Riddhi Kulkarni:** Rule-Based Engine, Elasticsearch and SMTP Setup.

5.5.2 Management reporting and communication

Regular reporting and efficient communication were essential to this project's successful completion. The following techniques and resources were employed to guarantee efficient coordination and progress monitoring:

- **Team Collaboration**
 - To monitor progress and talk about future tasks, weekly team meetings were held both online and offline.
 - A task board kept in a shared Google document was used to assign responsibilities to members with deadlines.
- **Guide Communication**
 - During review sessions, the internal guide received regular updates.
 - Weekly communication with the industry mentor took place, and the industry mentor received progress reports and demonstration snapshots.
- **Documentation and Reporting**
 - Following each team meeting, action items and meeting minutes were recorded.
 - The system architecture, code comments, and test results were kept in shared folders with other technical documentation.
- **Tools Used**
 - **GitHub:** Used for code collaboration and version control.
 - **Google Drive:** used for meeting notes, progress tracking sheets, and document sharing.

- **WhatsApp/Email:** For urgent coordination and prompt communication.
- **Google Meet:** Used for virtual team meetings and discussions with college professor.
- **Zoom Meetings:** Used for weekly communication with Industry Mentor
- **Reporting Frequency**
 - Internal team reports were discussed weekly.
 - Formal reporting to the external guide occurred weekly.
 - Major milestones and final results were reported to the internal guide as per requirements along with the review timetable as decided by the institute.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

Each of the several essential modules that make up the Network File System (NFS) anomaly detection system has a distinct purpose. These modules are discussed in detail in the following subsections:

1. **NFS File Access Logging:** Uses system logging tools like Elasticsearch and iNotify to record comprehensive logs of file operations (read, write, delete, and modify).
2. **Real-time Data Collection (Flask Server):** System logs and statistics from client computers are gathered by a Python-based backend and sent to analysis modules. This also exposes WebSocket channels for real-time communication with the web dashboard.
3. **Rule-based Detection Engine:** A lightweight engine that uses preset rules to identify common anomalies such as burst file modification, bulk deletion, and unauthorized access.
4. **Machine Learning Engine:** Identifies complex and unknown anomalous behaviors from the gathered data by using unsupervised learning models. Suitable for zero day vulnerabilities that show significant deviation from the usual user access patterns.
5. **Web Dashboard (React.js):** NFS administrators can view logs, alerts, anomalies, network packet statistics, and system health with real-time updates using a front-end interface.
6. **Alerting System (Elastalert2):** Uses Elasticsearch's output to generate real-time alerts for questionable activity. Using Elastalert2, notifications are sent via an email SMTP server to concerned employees and network administrators.
7. **Explainable AI summaries** Use of Gemini Model for creating a brief summary of the entire report which will be helpful in auditing, investigation as well as communications involving top level management.

6.2 Tools and Technologies Used

- **Frontend:** React.js for real-time interactive dashboards.
- **Backend:** Flask (Python) for API and server-side logic.

- **Authentication/Storage:** Appwrite for secure user and data management.
- **Log Collection:** iNotify Tools and Python with the Elasticsearch API for file access logging.
- **Log Analysis:** Elasticsearch for querying, Elastalert for alerting.
- **Machine Learning:** Python 3 with Scikit-learn for implementing ML algorithms (Isolation Forest, DBSCAN, K-Means). Gemini API for text summarization

6.3 Algorithm Details

The anomaly detection model integrates multiple unsupervised learning algorithms in a hybrid ensemble framework. Implementation involves systematic pre-processing, model application, and performance evaluation.

6.3.1 Isolation Forest

By choosing a feature at random and then choosing a split value at random from the feature's maximum and minimum values, the unsupervised anomaly detection algorithm Isolation Forest isolates observations.

- **Input:** Dataset D , number of trees t , subsample size s .
- **For** each tree in t :
 1. Randomly select a subsample of size s from D .
 2. Construct an isolation tree by recursively:
 - (a) Selecting a random feature.
 - (b) Selecting a random split value between the minimum and maximum values of the selected feature.
 - (c) Splitting the data into two parts based on the split value.
 - (d) Repeating the process for each partition until:
 - The node has only one instance, or
 - The tree reaches a predefined maximum height.
- **Compute** the average path length for each instance across all trees.
- **Calculate** the anomaly score based on the average path length.
- **Output:** Anomaly scores for each instance.

6.3.2 DBSCAN

A clustering algorithm called DBSCAN labels points that are isolated in low-density areas as outliers and groups points that are densely packed together.

- **Input:** Dataset D , distance threshold ε , minimum points $MinPts$.
- **Initialize:** Cluster label $C = 0$.
- **For** each unvisited point P in D :
 1. Mark P as visited.
 2. Retrieve the ε -neighborhood N of P .
 3. **If** $|N| < MinPts$:
 - Mark P as noise.
 4. **Else:**
 - Increment C .
 - Expand cluster C starting from P and its neighbors in N .
- **Output:** Cluster assignments for each point.

6.3.3 K-Means Clustering

A dataset is partitioned using the K-Means algorithm into k separate, non-overlapping clusters.

- **Input:** Dataset D , number of clusters k .
- **Initialize:** Randomly select k data points as initial centroids.
- **Repeat** until convergence:
 1. A dataset is partitioned using the K-Means algorithm into k separate, non-overlapping clusters.
 2. Recalculate the centroids as the mean of all points assigned to each cluster.
- **Output:** Final cluster assignments and centroids.

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

- **Unit Testing:** To guarantee expected behavior for specified inputs, separate modules, including the ML engine, rule engine, and log parser, were tested separately.
- **Integration Testing:** Confirmed how the Flask backend, React frontend, ML detection engine, rule engine, and logging service interacted. made sure that data flowed smoothly from anomaly alerts to NFS access.
- **Functional Testing:** Verified that the system accurately detects anomalies based on simulated and known behavioral patterns.
- **System Testing:** NFS client access logs are used for end-to-end system testing to verify general anomaly detection, logging, and visualization capabilities.
- **Performance Testing:** Evaluated the system's capacity to handle a high volume of logs in real time without any loss or lag.
- **User Acceptance Testing (UAT):** Made sure the system satisfies NFS administrators' requirements, emphasizing dashboard usability and alert efficacy.

7.2 Test Cases & Test Results

The test cases are divided based on major modules of the system:

7.2.1 UI Testing (Frontend)

- Log tables, charts, and alert notifications were rendered and verified.
- Tested live updates from the Flask server.

Table 7.1: UI Test Cases

Test Case ID	Description	Expected Output	Result
UI01	Render anomaly logs in table	Table displays data dynamically	Pass
UI02	Visualize file access graph	Line chart renders access count	Pass
UI03	Alert on rule match	Red alert banner shown	Pass
UI04	Unauthorized access to dashboard	Access denied message shown	Pass

7.2.2 Server and API Testing

- Verified if Flask server correctly receives and processes incoming logs.
- Ensured API endpoints return correct status codes and responses.

Table 7.2: Server Test Cases

Test Case ID	Description	Expected Output	Result
API01	POST log to Flask endpoint	200 OK with response message	Pass
API02	Malformed JSON in request	400 Bad Request	Pass
API03	Realtime Socket connection	Connection established successfully	Pass

7.2.3 Machine Learning Engine Testing

- Tested accuracy and behavior of Isolation Forest, DBSCAN, and K-Means.
- Evaluated outputs using metrics like Silhouette Score and Davies-Bouldin Index.

Table 7.3: ML Engine Test Cases

Test Case ID	Description	Expected Output	Result
ML01	Anomaly detection by Isolation Forest	Outlier marked as -1	Pass
ML02	DBSCAN noise detection	Noise points marked -1	Pass
ML03	Ensemble logic output	Anomaly if any model detects it	Pass
ML04	Evaluate clustering quality	Silhouette Score > 0.4	Pass

7.2.4 Rule-Based Engine Testing

- Verified that static rules fire based on log attributes (file extension, access type, etc.).
- Ensured alert messages are generated correctly and sent to UI.

Table 7.4: Rule-Based Engine Test Cases

Test Case ID	Description	Expected Output	Result
RB01	Attempt unauthorized file creation	Rule triggered, alert raised	Pass
RB02	High modification frequency	Detection rule activated	Pass
RB03	Suspicious file extension (.swp)	Alert generated for file	Pass

CHAPTER 8

RESULTS

8.1 Outcomes

8.1.1 Performance Metrics

To assess the performance of clustering algorithms, two widely used metrics are the **Silhouette Score** and the **Davies-Bouldin Index**. These metrics help to quantify how well the clusters are formed, considering both the cohesion within clusters and the separation between clusters.

- **Silhouette Score:** The degree to which each point resembles its own cluster in relation to other clusters is indicated by the Silhouette Score. The Score has a range from -1 to 1 , where a score close to 1 indicates that the data points are well clustered.
- **Davies-Bouldin Index:** The Davies-Bouldin Index (DBI) is based on the average similarity ratio of each cluster with the cluster that is most similar to it.

Better clustering is indicated by a lower DBI since it implies that the clusters are compact and widely spaced. The clusters are either overlapping or excessively dispersed when the DBI is higher.

Evaluation Metrics	DBSCAN	Isolation Forest	K-Means
Silhouette Score	0.9988120	0.9397055	0.9493090
Davies-Bouldin Index	0.0343131	0.0833411	0.0094942

Table 8.1: ML Model Scores

8.1.2 Real-time Processing

The system works perfectly in real-time. The user access pattern was simulated using bash scripts by connecting multiple clients from distinct IP addresses. Access logs were generated and captured in parallel, and anomalies were detected in real-time. The console reflected these alerts immediately with the help of web-socket.

The system statistics also updated in real-time, showing CPU, memory, disk, and network usage.

8.1.3 System Improvement

The custom python logger helped in capturing important features such as client IP and file paths, simultaneously. It bridged the gap in available logging tool's performance.

The iterative testing helped improve web console experience in ways such as:

- Optimizing API calls for faster and periodic delivery.
- UI changes to easily identify suspicious and high priority alerts.
- Data security policies were added to ensure industry compliance.

8.1.4 User Feedback

We received valuable feedback from industry experts who used the product and suggested future enhancements such as:

- In cyber security, balancing precision and recall is crucial - too high precision may miss certain threats, while too high recall increases false positives.
- Rule-based engines tend to produce false positives, especially in predefined security policies. The model should adapt based on user feedback.
- Instead of applying a one-size-fits-all model, it should continuously refine its understanding based on organizational and user-specific behavior.

8.2 Screen Shots

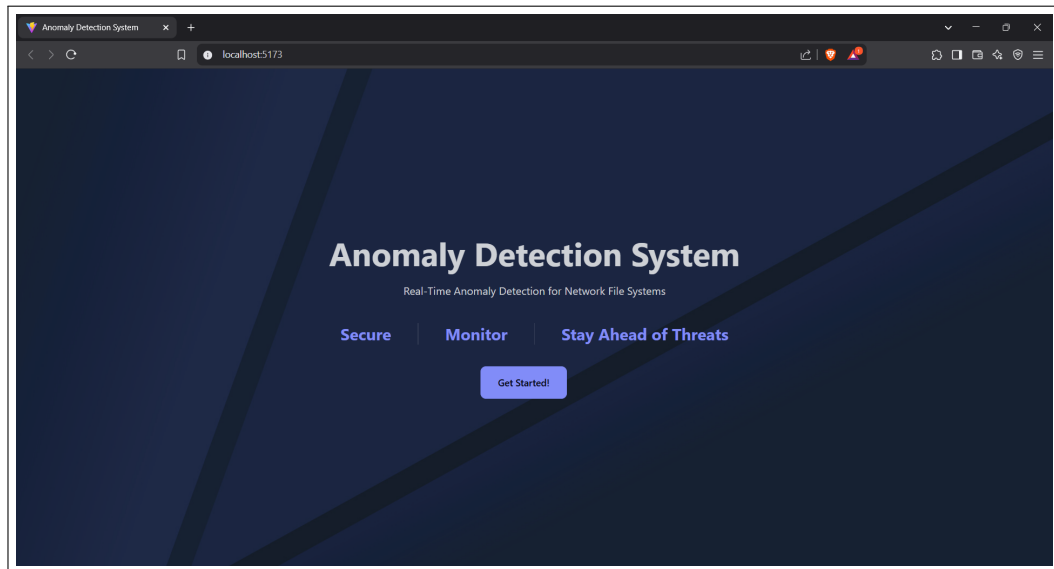


Figure 8.1: Landing Page

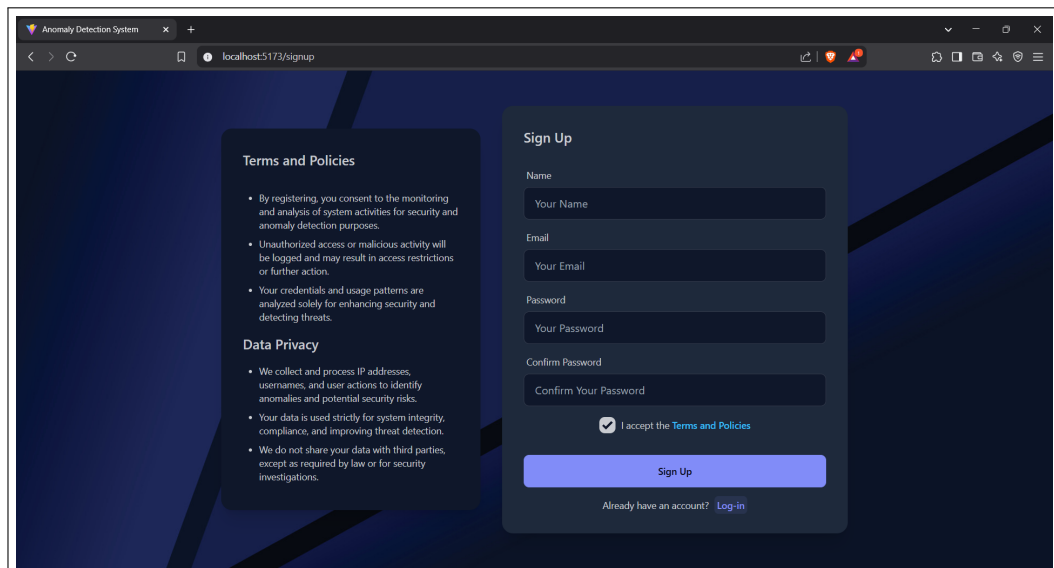


Figure 8.2: Sign-up Page

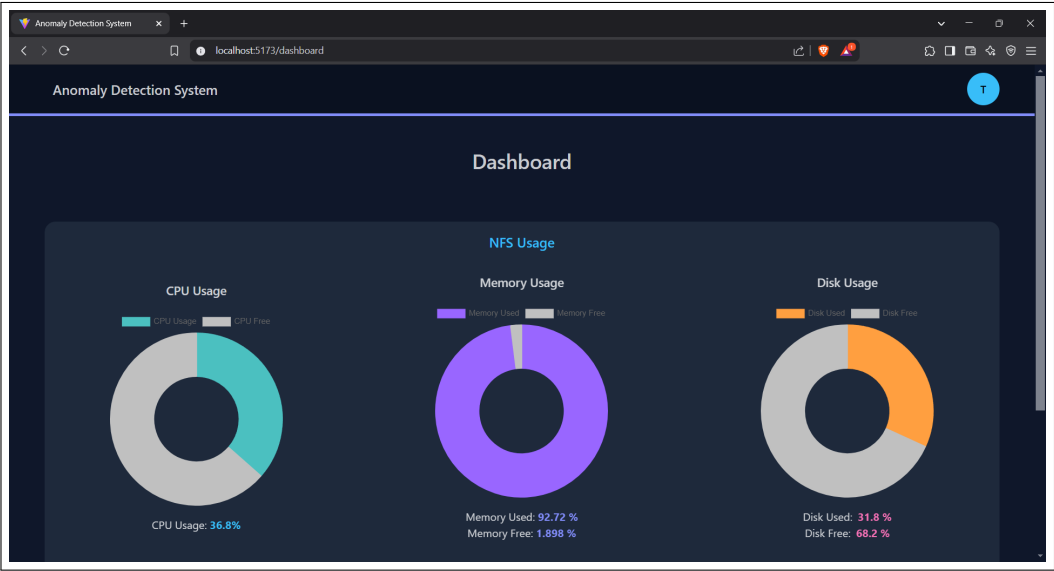


Figure 8.3: Dashboard: System Statistics

Live Rule-based Alerts

Rule Name: All Rules Operation: All Operations File Path: File Path ID: Log ID

Rule Name	Start Time	End Time	File Path	IP Address	Operation	ID
Suspicious File Extension Alert	2025-03-20T17:42:48.165053Z	2025-03-21T04:43:27.41781Z	/mnt/nfs_shared/16.py	192.168.65.3	DELETED	lueltUUB...
Suspicious File Extension Alert	2025-03-20T17:42:48.165053Z	2025-03-21T04:43:27.41781Z	/mnt/nfs_shared/17.py	192.168.65.3	DELETED	I-eltUUB...
Suspicious File Extension Alert	2025-03-20T17:42:48.165053Z	2025-03-21T04:43:27.41781Z	/mnt/nfs_shared/18.py	192.168.65.3	DELETED	mOeltUUB...
Suspicious File Extension Alert	2025-03-20T17:42:48.165053Z	2025-03-21T04:43:27.41781Z	/mnt/nfs_shared/19.py	192.168.65.3	DELETED	meeltUUB...
Suspicious File Extension	2025-03-	2025-03-	...	...	...	...

Export As 1 / 25

Figure 8.4: Alerts: Rule Based Engine

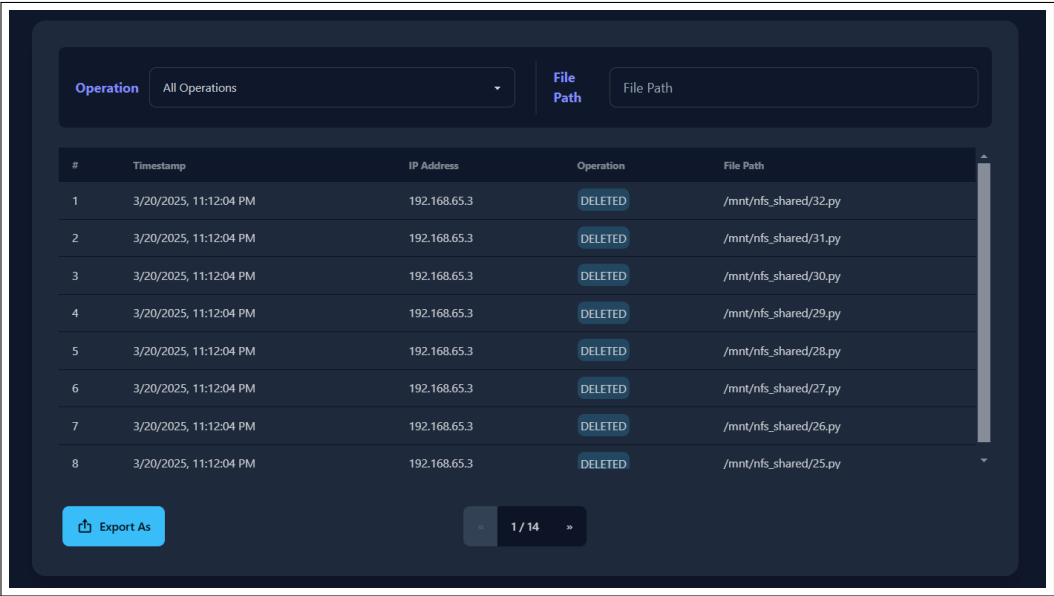


Figure 8.5: Alerts: ML Based Engine



Figure 8.6: Visualization Charts



Figure 8.7: Explainable AI (Summary of the Anomalies)

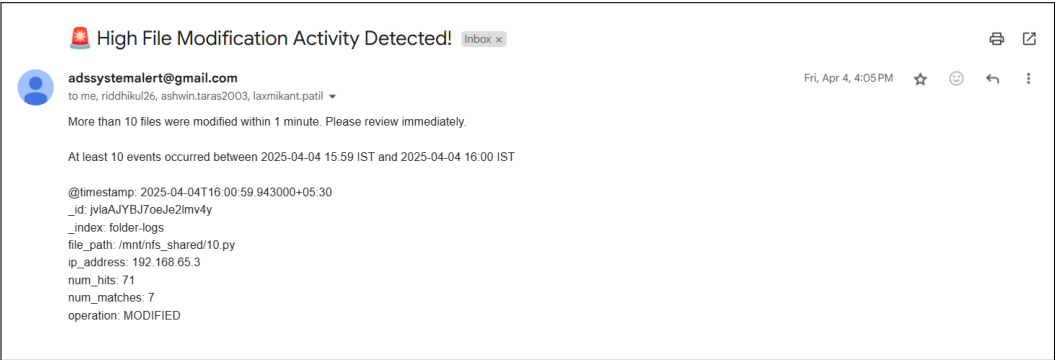


Figure 8.8: Example of Alert Email

CHAPTER 9

CONCLUSIONS

9.1 Conclusion

Our anomaly detection solution leverages the power of Unsupervised Machine Learning algorithms to intelligently monitor user activities and system operations within file systems. By analyzing system logs and user behavior patterns, the model identifies deviations that could indicate potential security threats, unauthorized access, or system inefficiencies.

Through a user-friendly dashboard providing near real-time insights, NFS administrators and security teams are empowered to proactively respond to anomalies, thereby safeguarding sensitive data and maintaining operational stability. The solution not only enhances resource management but also significantly strengthens the security framework by minimizing both false positives and false negatives.

Furthermore, the system's modular architecture, comprising Flask-based servers, Elasticsearch databases, and scalable machine learning components, ensures that it can evolve with future requirements, including cloud migration and enterprise-level integrations. In conclusion, this project delivers a reliable, scalable, and efficient approach to anomaly detection, bridging the gap between traditional monitoring methods and modern intelligent security solutions.

9.2 Future Work

- Future research will focus on improving detection accuracy through the use of more sophisticated machine learning methods, like deep learning models, to examine intricate access patterns.
- Furthermore, expanding the tool's capabilities to support more detailed anomaly classifications and make it compatible with a range of NAS platforms will increase its accuracy and adaptability.
- System security and resource management will be further optimized by integrating automated remediation mechanisms like dynamic access control and alert-triggered responses.
- Additionally, the tool will be more appropriate for businesses handling increasingly complex data as efforts are made to increase its scalability for handling larger datasets and multi-NAS environments.

9.3 Applications

1. **NFS Administrators:** The system assists Network File System (NFS) administrators in continuously monitoring user activities on file servers. By identifying irregular access patterns and potential data misuse, it ensures data integrity, enforces role-based access policies, and proactively prevents unauthorized operations within shared storage environments.
2. **Security and Cybersecurity Teams:** Security personnel can leverage the anomaly detection system to identify insider threats, malware activities, and unauthorized access attempts. The system serves as an early warning tool, enhancing the organization's overall security posture by detecting anomalies before they escalate into major security incidents.
3. **IT Operations and System Administrators:** IT teams can utilize the insights generated to optimize file system performance, detect bottlenecks in storage access, and manage system resources more efficiently. Additionally, historical anomaly data helps in planning infrastructure scaling and performing preventive maintenance.
4. **Compliance and Audit Teams:** Regulatory compliance officers and auditors can use the system to generate evidence of secure file access practices, helping organizations meet standards such as ISO 27001, HIPAA, or GDPR by identifying unauthorized data usage.
5. **Incident Response Teams:** During and after security incidents, the system provides forensic insights into anomalous file access behavior, enabling faster root cause analysis and assisting in post-incident reporting and system recovery.
6. **System Architects and Engineers:** Data collected from anomaly detection can be used by architects and system designers to refine access control policies, improve infrastructure security, and develop more resilient storage and access architectures.

CHAPTER 10

APPENDIX

10.1 Appendix A: Configuration Details

This appendix provides information about system setup, configuration files, and dependencies used in the project.

10.1.1 System Configuration

- **Operating System:** Ubuntu 20.04 LTS
- **Backend Framework:** Flask (Python 3.10)
- **Frontend Framework:** React.js with Tailwind CSS
- **Database:** Appwrite
- **Log Storage and Search:** Elasticsearch 8.x
- **Alert System:** Elastaalert2
- **Visualization Tools:** Plotly, Chart.js
- **Virtualization:** Docker (for containerization of modules)
- **NFS Configuration:** Configured with `/etc/exports` for shared directories across Docker containers

10.1.2 Environment Variables

```
FLASK_ENV=development
ELASTICSEARCH_HOST=http://localhost:9200
SMTP_SERVER=smtp.gmail.com
SMTP_PORT=587
```

10.1.3 Sample Log Format

Below is the sample format of collected logs used for anomaly detection:

```
{
  "timestamp": "2025-03-15T14:05:23Z",
  "user": "ashwin_taras",
  "operation": "write",
  "file_path": "/mnt/nfs/shared/reports/data.csv",
  "client_ip": "192.168.1.20"
}
```

10.1.4 Explanation of Fields

- **timestamp:** UTC time of the operation.
- **user:** System username executing the operation.
- **operation:** Type of operation (read/write/delete).
- **file_path:** Full path to the file.
- **client_ip:** IP address of the machine performing the action.

10.1.5 Usage Notes

These configurations and sample logs represent a typical deployment environment for testing anomaly detection in a simulated NFS scenario. Logs are collected in real-time and passed through both rule-based and machine learning filters before being visualized or triggering alerts.

10.2 Appendix B: Publishing Details

10.2.1 Survey Paper

- **Title:** Overview of Machine Learning-Based Anomaly Detection in Linux-Based File Systems
- **Journal:** 7th International Conference on Communication and Computational Technologies (ICCCT 2025)
- **Status:** Presented (ICCCT, February 2025)

10.2.2 Implementation Paper

- **Title:** Anomaly Detection in Network File System
- **Journal:** BJIT - BVICAM's International Journal of Information Technology
- **Status:** Submitted

10.3 Appendix C: Plagiarism Report

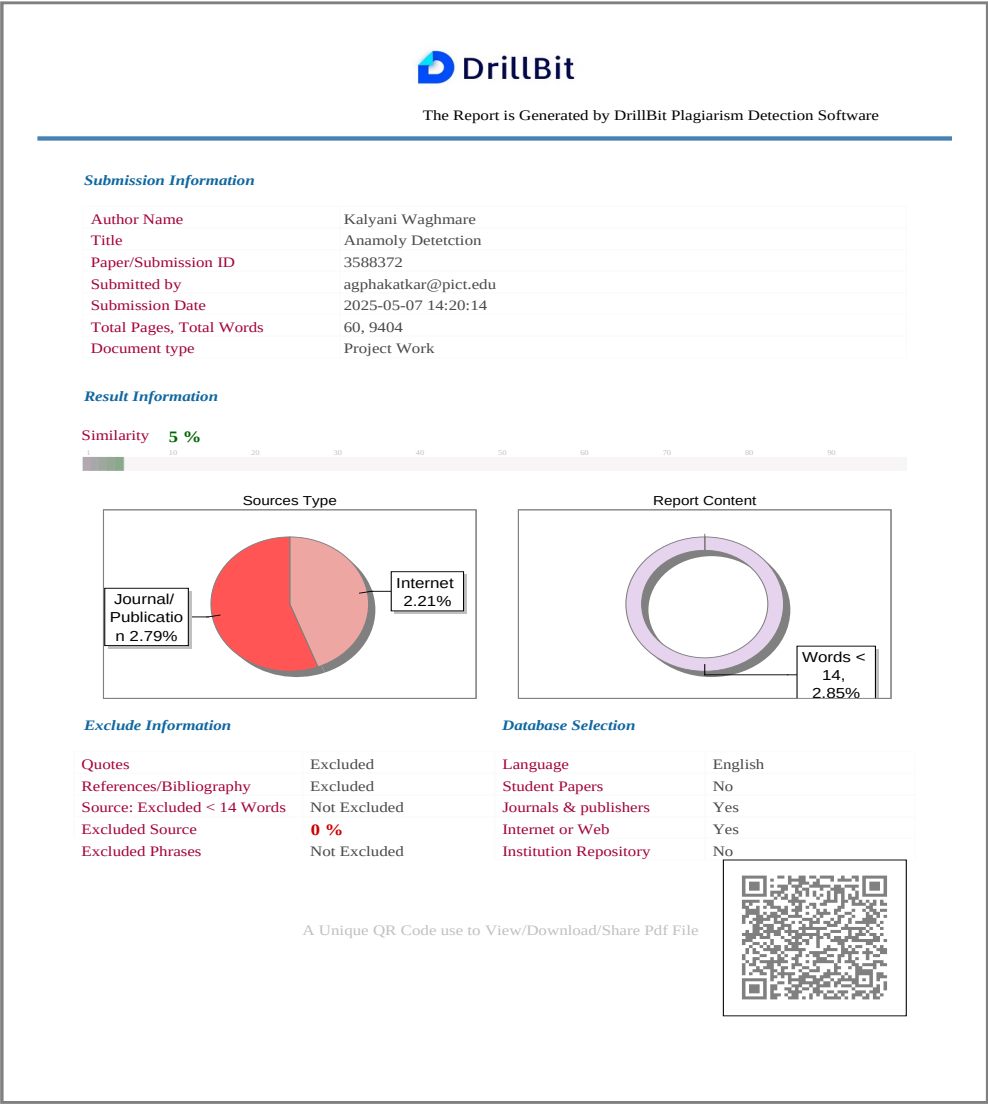


Figure 10.1: Plagiarism Report

CHAPTER 11

REFERENCES

- [1] Goldstein, M., & Dengel, A. (2012). Histogram-based outlier score (HBOS): A fast unsupervised anomaly detection algorithm. In *Proceedings of the International Conference on Big Data Analytics and Knowledge Discovery* (pp. 1–10).
- [2] Sharma, A., & Kaushal, A. (2016). Histogram-based anomaly detection in network traffic. *Journal of Computer Networks and Communications*, 2016, 1–12.
- [3] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining* (pp. 413–422). IEEE.
- [4] Farzad, A., & Gulliver, T. A. (2020). Unsupervised log message anomaly detection. *ICT Express*, 6(3), 229–237.
- [5] Çelik, M., & Dadaşer-Çelik, F. (2011). Anomaly detection in temperature data using DBSCAN algorithm. In *Proceedings of the International Symposium on Innovations in Intelligent Systems and Applications* (pp. 1–5).
- [6] Jia, Y., Chen, H., Yuan, L., & Hou, X. (2021). Flight operation anomaly detection based on one class SVM. In *Proceedings of the Fifth International Conference on Traffic Engineering and Transportation System (ICTETS 2021)* (Vol. 12058, p. 120583A). SPIE.
- [7] Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: Identifying density-based local outliers. *SIGMOD Record*, 29, 93–104.
- [8] Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM Conference on Computer and Communications Security (CCS'17)* (pp. 1285–1298). ACM.
- [9] Meng, W., Liu, Y., Zhu, Y., Zhang, S., Pei, D., Liu, Y., Chen, Y., et al. (2019). LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI'19)* (pp. 4739–4745).
- [10] Nedelkoski, S., Bogatinovski, J., Acker, A., Cardoso, J., & Kao, O. (2020). Self-attentive classification-based anomaly detection in unstructured logs. *arXiv preprint arXiv:2006.03989*.

List of Abbreviations

Abbreviation	Full Form
ML	Machine Learning
NFS	Network File System
UI	User Interface
UAT	User Acceptance Testing
RBAC	Role-Based Access Control
API	Application Programming Interface

List of Figures

Figure	Title	Page
4.1	System Architecture: Anomaly Detection System	24
4.2	System Architecture with Multiple Clients	24
4.3	State Transition Diagram for Mathematical Model	26
4.4	Data Flow Diagram (DFD-0)	27
4.5	Data Flow Diagram (DFD-1)	27
4.6	Data Flow Diagram (DFD-2)	28
4.7	Entity Relationship Diagram	29
4.8	Use Case Diagram	30
4.9	Sequence Diagram	31
4.10	Deployment Diagram	32
5.1	Phase 1: Research and Analysis	43
5.2	Phase 2: Implementation and Deployment	43
8.1	Landing Page	58
8.2	Sign-up Page	58
8.3	Dashboard: System Statistics	59
8.4	Alerts: Rule Based Engine	59
8.5	Alerts: ML Based Engine	60
8.6	Visualization Charts	60
8.7	Explainable AI (Summary of the Anomalies)	61
8.8	Example of Alert Email	61

List of Tables

Table	Title	Page
4.1	Project Plan	29
5.1	Risk ID 1: Log Collection Failure	36
5.2	Risk ID 2: ML Model Inaccuracy	37
5.3	Risk ID 3: False Positives from Rules	38
5.4	Risk ID 4: Dashboard Render Failure	39
5.5	Task Dependency Table	42
7.1	UI Test Cases	51
7.2	Server Test Cases	52
7.3	ML Engine Test Cases	53
7.4	Rule-Based Engine Test Cases	54
8.1	ML Model Scores	56